

## TASK 1 — Process Creation and Command Execution

```
krishnasree08@krishnasree08:~$ mkdir practical_week2
krishnasree08@krishnasree08:~$ cd practical_week2
krishnasree08@krishnasree08:~/practical_week2$ pwd
/home/krishnasree08/practical_week2
krishnasree08@krishnasree08:~/practical_week2$ nano command_executor.c
krishnasree08@krishnasree08:~/practical_week2$ gcc command_executor.c -o command_executor
krishnasree08@krishnasree08:~/practical_week2$ ls
command_executor  command_executor.c
krishnasree08@krishnasree08:~/practical_week2$ ./command_executor
Enter a Linux command: ls

Parent Process

Child Process
Parent PID : 490
Child PID  : 491
Child PID  : 491
Parent PID : 490

Executing command: ls
command_executor  command_executor.c

Child process completed.
krishnasree08@krishnasree08:~/practical_week2$ ./command_executor
Enter a Linux command: pwd

Parent Process
Parent PID : 493
Child PID  : 494

Child Process
Child PID  : 494
Parent PID : 493

Executing command: pwd
/home/krishnasree08/practical_week2

Child process completed.
```

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <string.h>

int main()
{
    char command[100];

    printf("Enter a Linux command: ");
    fgets(command, sizeof(command), stdin);

    command[strcspn(command, "\n")] = '\0';

    pid_t pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        return 1;
    }

    else if (pid == 0)
    {
        printf("\nChild Process\n");
        printf("Child PID : %d\n", getpid());
        printf("Parent PID : %d\n", getppid());

        printf("\nExecuting command: %s\n", command);

        execl("/bin/sh", "sh", "-c", command, (char *)NULL);

        perror("exec failed");
        exit(1);
    }
}
```

```

else
{
    printf("\nParent Process\n");
    printf("Parent PID : %d\n", getpid());
    printf("Child PID  : %d\n", pid);

    wait(NULL);

    printf("\nChild process completed.\n");
}

return 0;
}

```

### Summary:

The program demonstrates how Linux creates and manages processes. It accepts a Linux command from the user, creates a child process using `fork()`, executes the entered command in the child process using an `exec()` system call, and makes the parent process wait using `wait()`. The program also displays the Process IDs (PIDs) of both the parent and child processes.

### Result:

Thus, the Linux process creation and command execution using `fork()`, `exec()`, and `wait()` system calls was successfully implemented.

### TASK 2 — Hardware Resources and OS Services

```

krishnasree08@krishnasree08:~/practical_week2$ uname
Linux
krishnasree08@krishnasree08:~/practical_week2$ uname -a
Linux krishnasree08 6.18.33.2-microsoft-standard-WSL2 #1 SMP PREEMPT_DYNAMIC Thu Jun 18 21:54:43 UTC 2026 x86_64 x86_64 x86_64 GNU/Linux

```

```

Architecture:            x86_64
CPU op-mode(s):          32-bit, 64-bit
Address sizes:            48 bits physical, 48 bits virtual
Byte Order:               Little Endian
CPU(s):                   16
On-line CPU(s) list:      0-15
Vendor ID:                 AuthenticAMD
Model name:                AMD Ryzen 7 7730U with Radeon Graphics
CPU family:                25
Model:                    80
Thread(s) per core:        2
Core(s) per socket:        8
Socket(s):                 1
Stepping:                  0
BogoMIPS:                  3992.40

```

```

krishnasree08@krishnasree08:~/practical_week2$ ps
  PID TTY          TIME CMD
  293 pts/0        00:00:00 bash
  529 pts/0        00:00:00 ps

```

```
krishnasree08@krishnasree08:~/practical_week2$ top
top - 06:41:27 up 8 min, 1 user, load average: 0.00, 0.02, 0.00
Tasks: 24 total, 1 running, 23 sleeping, 0 stopped, 0 zombie
%Cpu(s): 0.0 us, 0.0 sy, 0.0 ni,100.0 id, 0.0 wa, 0.0 hi, 0.0 si, 0.0 st
MiB Mem : 6849.7 total, 5500.2 free, 564.4 used, 933.4 buff/cache
MiB Swap: 2048.0 total, 2048.0 free, 0.0 used. 6285.3 avail Mem

  PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM    TIME+    COMMAND
    1 root        20   0   21528   13116   9792 S   0.0   0.2   0:00.68 systemd
    2 root        20   0    3180    2212   2072 S   0.0   0.0   0:00.00 init-systemd(Ub
    6 root        20   0    3196    2128   2008 S   0.0   0.0   0:00.00 init
   52 root        19  -1   50308   16096  15024 S   0.0   0.2   0:00.21 systemd-journal
  100 root        20   0   25284    6612   5168 S   0.0   0.1   0:00.27 systemd-udevd
  117 systemd+    20   0   21344   13224  11104 S   0.0   0.2   0:00.11 systemd-resolve
  119 systemd+    20   0   91036   7988   7024 S   0.0   0.1   0:00.10 systemd-timesyn
  180 root        20   0    4244    2688   2432 S   0.0   0.0   0:00.01 cron
  181 message+    20   0    9592    5340   4664 S   0.0   0.1   0:00.07 dbus-daemon
  188 root        20   0   17976    8832   7796 S   0.0   0.1   0:00.09 systemd-logind
  190 root        20   0  1755848  13060  10888 S   0.0   0.2   0:00.14 wsl-pro-service
  199 syslog      20   0   222516   6028   4644 S   0.0   0.1   0:00.07 rsyslogd
  201 root        20   0    3124    1972   1836 S   0.0   0.0   0:00.00agetty
  221 root        20   0  107032   23308  13848 S   0.0   0.3   0:00.13 unattended-upgr
  287 root        20   0    3188    1116    980 S   0.0   0.0   0:00.00 SessionLeader
  288 root        20   0    3204    1252   1108 S   0.0   0.0   0:00.05 Relay(293)
  293 krishna+    20   0    6080    5384   3664 S   0.0   0.1   0:00.05 bash
  296 root        20   0    6700    4644   3860 S   0.0   0.1   0:00.00 login
  356 krishna+    20   0   20328   11572   9440 S   0.0   0.2   0:00.11 systemd
  357 krishna+    20   0   21164    3572   1848 S   0.0   0.1   0:00.00 (sd-pam)
  379 krishna+    20   0    6080    5340   3676 S   0.0   0.1   0:00.02 bash
  531 krishna+    20   0    9300    5676   3488 R   0.0   0.1   0:00.01 top
  532 root        20   0   25288    3696   2216 S   0.0   0.1   0:00.00 (udev-worker)
  533 root        20   0   25288    3696   2216 S   0.0   0.1   0:00.00 (udev-worker)
```

```
krishnasree08@krishnasree08:~/practical_week2$ q
Command 'q' not found, but can be installed with:
sudo snap install q # version 1.6.3-1, or
sudo apt install python3-q-text-as-data # version 3.1.6-3
See 'snap info q' for additional versions.
```

### Summary:

This task investigates the relationship between hardware resources and operating system services using Linux terminal commands. The `uname` command provides information about the Linux operating system and kernel, `lscpu` displays CPU and processor details, `ps` displays information about running processes, and `top` provides real-time information about CPU, memory, and process utilization. These commands demonstrate how the operating system manages and provides access to hardware resources.

### Result:

Thus, the hardware resources and operating system services were successfully investigated using `uname`, `lscpu`, `ps`, and `top` commands.

